

Multimodal Manufacturing Creator

Course Name: Generative AI

Institution Name: *Medicaps University – Datagami Skill Based Course*

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aaditya Shrivastava	EN22CS301007
2.	Aakansha Jaiswal	EN22CS301011
3.	Akanksha Sharma	EN22CS301080
4.	Akshita Bhawsar	EN22CS301098
5.	Abhishek Malviya	EN22CS301034

Group Name: Group 12D1

Project Number: GAI-12

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

The manufacturing industry increasingly relies on rapid concept visualization to communicate product ideas, streamline prototyping decisions, and support educational demonstrations. However, translating a textual manufacturing idea into both a detailed technical description and a visual representation traditionally requires significant domain expertise, design tools, and considerable time investment.

Existing tools address either text generation or image generation in isolation, creating a fragmented workflow for engineers, students, and educators. There is a clear gap for an integrated, AI-powered solution that can take a simple natural language prompt and simultaneously produce a structured technical narrative alongside a photorealistic visual prototype all within seconds.

This project addresses that gap by building the Multimodal Manufacturing Creator: a GenAI application that combines a Large Language Model (LLM) with an image generation model, enabling seamless, multimodal content creation for manufacturing concept visualization

2. Project Objectives

- Develop a multimodal GenAI application that accepts natural language manufacturing prompts and generates both descriptive narratives and visual prototypes
- Integrate OpenAI GPT-4o-mini for structured text generation covering product description and material specifications
- Integrate OpenAI DALL·E 3 for generating photorealistic, high-fidelity visual representations of manufacturing concepts
- Implement a local vector database (ChromaDB) to store, retrieve, and perform semantic similarity search on previously generated concepts
- Build a clean, professional web interface using Streamlit with generation progress tracking and download capabilities
- Demonstrate the synergy between text-to-text and text-to-image models in a real-world manufacturing education context

3. Scope of the Project

This project covers the following areas:

- Natural language to structured narrative generation for manufacturing concepts
- Automated image prompt optimization and visual prototype generation
- Local vector-based knowledge base for concept storage and similarity retrieval
- A fully functional Streamlit web application deployable on any local machine

- Cost-effective implementation using a single API provider (OpenAI)

The project does not cover: real-time manufacturing simulation, CAD file generation, multi-user collaboration, or cloud deployment. These are identified as future enhancements.

Proposed Solution

The Multimodal Manufacturing Creator is a Python-based web application that integrates two AI modalities language and vision into a single cohesive pipeline. A user enters a manufacturing concept in plain English, and the system produces both a structured technical narrative and a photorealistic visual representation, while storing the output in a local vector database for future reference and similarity search.

1. Key features

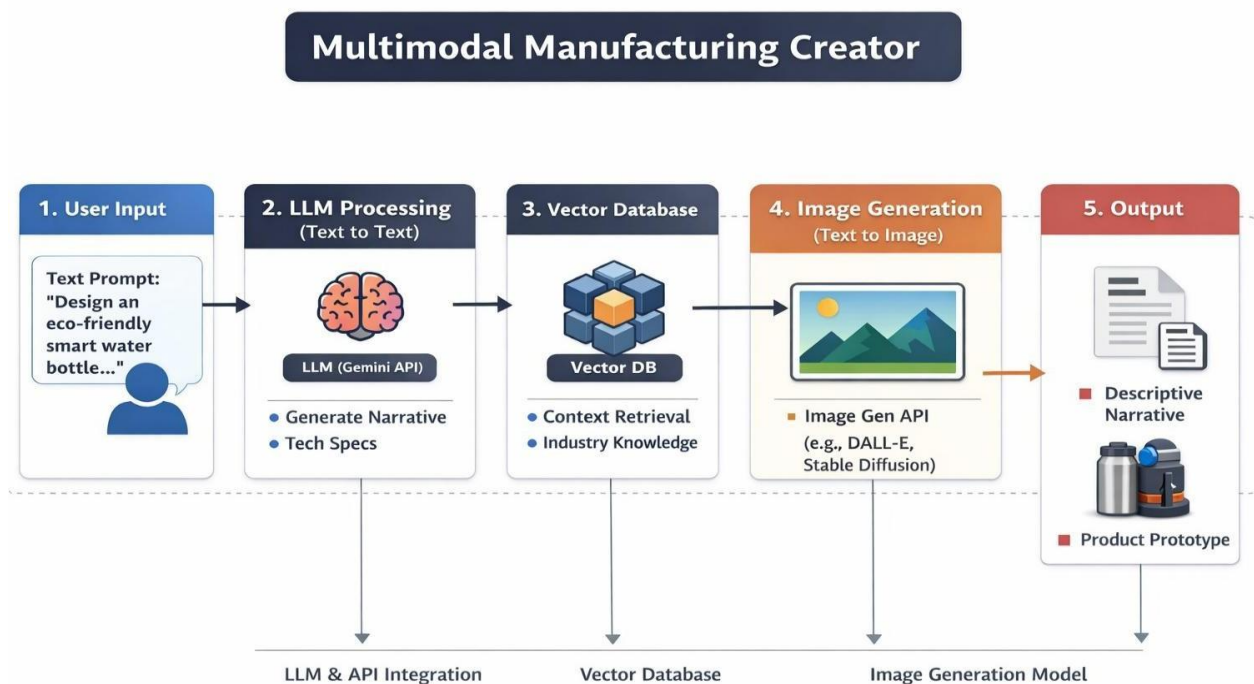
- **Multimodal Generation:** Simultaneously produces text narratives and visual images from a single prompt
- **Optimized Image Prompting:** GPT-4o-mini automatically refines user prompts for better DALL·E 3 outputs
- **Structured Narratives:** Responses follow a consistent format Product Description + Materials keeping output concise and professional
- **Semantic Search:** Before generating, the system searches the knowledge base for similar past concepts to avoid duplicate work
- **History Gallery:** A dedicated tab displays all previously generated concepts with expandable narratives and saved images
- **Download Support:** Users can download both the narrative (.txt) and the generated image (.png) directly from the UI
- **Real-Time Progress Bar:** Step-by-step progress feedback keeps users informed during the generation pipeline
- **Cost Controls:** Token limit slider and image quality selector allow users to balance quality vs. API cost

2. Overall Architecture / Workflow

The Multimodal Manufacturing Creator is a Python-based web application that accepts natural language descriptions of manufacturing concepts and produces two complementary outputs: a structured textual narrative and a photorealistic visual representation. The system leverages OpenAI's GPT-4o-mini for text generation and DALL·E 3 for image generation, with ChromaDB serving as a local vector database to enable semantic search and retrieval of previously generated concepts. The application follows a modular, layered architecture with five components working in sequence:

Component	Description
Frontend	Streamlit web interface with professional UI, tabs, and sidebar controls
LLM Module	GPT-4o-mini for generating manufacturing narratives and optimized image prompts
Image Module	DALL·E 3 for generating high-fidelity visual representations of concepts
Vector DB	ChromaDB (local) for storing and retrieving concept embeddings
Embeddings	OpenAI text-embedding-ada-002 for semantic vector representations

Workflow :



Step 1: User Input Layer

The workflow begins when the user provides a manufacturing idea in natural language format.

For example: "Design an eco-friendly smart water bottle for industrial workers."

This input is captured by the FastAPI backend and passed to the AI processing layer. The prompt becomes the foundation for both:

- Narrative generation
- Image generation

Step 2: LLM Processing (Text-to-Text)

The prompt is processed using the GPT-4o-mini(LLM) Large Language Model. The LLM converts the idea into:

- a) A structured product description
- b) Technical specifications
- c) Material suggestions
- d) Use-case explanation

This stage transforms an abstract idea into meaningful engineering

Step 3: Data Storage Layer (Vector Database)

LLM alone gives general answers. Vector DB enables Retrieval-Augmented Generation. The Vector Database enhances the intelligence of the system by retrieving domain-specific knowledge. This ensures that the generated manufacturing concept.

Step 4: Image Generation Layer

The structured description is then passed to the image generation Model (DALL-E3). The architecture is designed to integrate real text-to-image models such as Stable Diffusion or DALL·E. It generates:

- Prototype visualization
- Concept design image

Step 5: Output Layer

Finally, the system returns both:

- A descriptive narrative
- A product prototype visualization

3. Tools & Technologies Used

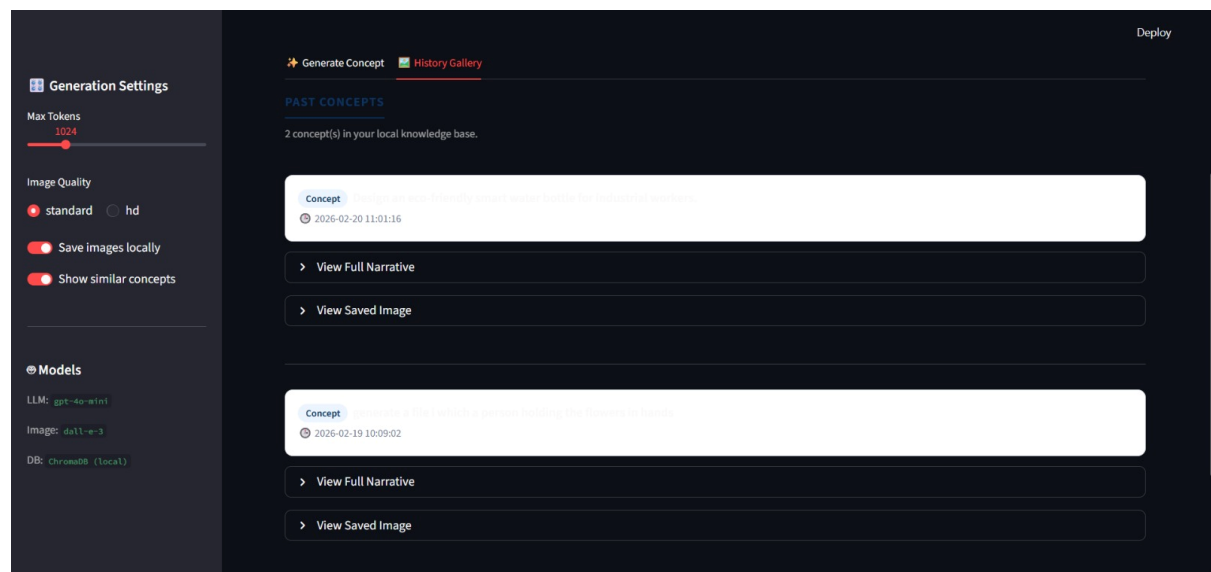
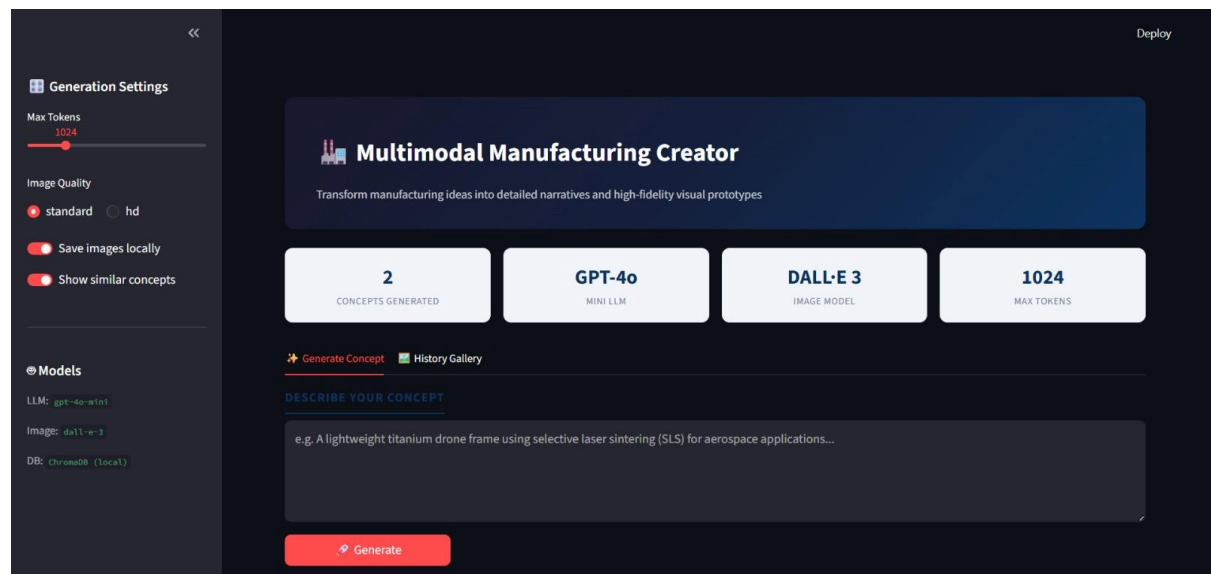
Category	Tool / Technology	Purpose
Programming Language	Python 3.10+	Core application development
LLM API	OpenAI GPT-4o-mini	Narrative generation and prompt optimization
Image Generation API	OpenAI DALL·E 3	Photorealistic visual prototype generation
Embeddings	text-embedding-ada-002	Semantic vector representation of concepts
Vector Database	ChromaDB (local)	Concept storage and similarity search
Web Framework	Streamlit	Interactive web UI with no frontend code required
Environment Config	python-dotenv	Secure API key management via .env file
HTTP Requests	requests	Downloading DALL·E images from URL to local disk
Version Control	Git / GitHub	Source code management and collaboration
IDE	VS Code	Development environment

Results & Output

The Multimodal Manufacturing Creator was successfully developed and tested with a range of manufacturing concept prompts. The system consistently produced high-quality structured narratives and photorealistic visual representations within 15–30 seconds per generation.

1. Screenshots

User Interface:



API Key Handling:

```

EXPLORER  ...  config.py 1  app.py  llm_handler.py
MANUFACTURING-CREATOR
  > _pycache_
  > data
  > modules
    > _pycache_
    embeddings.py
    image_handler.py
    llm_handler.py
    vector_db.py
  > Multimodal-Manufacturing
  > venv
  .env
  app.py
  config.py
  requirements.txt

config.py 1
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 # --- API Key (OpenAI only) ---
7 # OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
8
9 # --- Vector DB ---
10 CHROMA_DB_PATH = os.getenv("CHROMA_DB_PATH", "../data/chroma_db")
11
12 # --- LLM Settings ---
13 LLM_MODEL = "gpt-4o-mini"
14 MAX_TOKENS = int(os.getenv("MAX_TOKENS", 1024))
15
16 # --- Image Generation Settings ---
17 IMAGE_MODEL = "dall-e-3"
18 IMAGE_SIZE = os.getenv("IMAGE_SIZE", "1024x1024")
19 IMAGE_QUALITY = os.getenv("IMAGE_QUALITY", "standard") # "hd" costs more
20
21 # --- Embeddings ---
22 EMBEDDING_MODEL = "text-embedding-ada-002"
23
24 # --- App ---
25 APP_TITLE = os.getenv("APP_TITLE", "Multimodal Manufacturing Creator")
26
27
28 def validate_keys():
29     """Check that required API key is set."""
30     if not OPENAI_API_KEY:
31         raise EnvironmentError(
32             "Missing OPENAI_API_KEY.\n"
33             "Please copy .env.example -> .env and add your OpenAI API key."
34         )
35

```

LLM handling code:

```

EXPLORER  ...  config.py 1  app.py 1  llm_handler.py X
MANUFACTURING-CREATOR
  > _pycache_
  > data
  > modules
    > _pycache_
    embeddings.py
    image_handler.py
    llm_handler.py
    vector_db.py
  > Multimodal-Manufacturing
  > venv
  .env
  app.py
  config.py
  requirements.txt

modules > llm_handler.py > generate_image_prompt
1 from openai import OpenAI
2 import config
3 client = OpenAI(api_key=config.OPENAI_API_KEY)
4 SYSTEM_PROMPT = """You are a manufacturing expert. When given a manufacturing concept, respond in this exact format:
5 **Description**
6 Write 2-3 sentences describing what the product/concept is and what it does.
7 **Materials**
8 List 3-5 key materials used, each with a one-line explanation of why it's used.
9 Keep it concise and professional."""
10 def generate_narrative(user_prompt: str) -> str:
11     try:
12         response = client.chat.completions.create(
13             model=config.LLM_MODEL,
14             max_tokens=config.MAX_TOKENS,
15             messages=[
16                 ("role": "system", "content": SYSTEM_PROMPT),
17                 ("role": "user", "content": f"Generate a concept overview for: {user_prompt}")
18             ]
19         )
20         return response.choices[0].message.content
21     except Exception as e:
22         raise RuntimeError(f"LLM API error: {str(e)}")
23 def generate_image_prompt(user_prompt: str, narrative: str) -> str:
24     try:
25         response = client.chat.completions.create(
26             model=config.LLM_MODEL,
27             max_tokens=200,
28             messages=[
29                 ("role": "system", "content": "Write concise, vivid DALL-E image prompts focused on visual details."),
30                 ("role": "user", "content": (
31                     f"Write a DALL-E prompt (max 150 words) for: '{user_prompt}'\n"
32                     f"Context: {narrative[:400]}\n"
33                     "Focus on photorealistic appearance, materials, factory/lab setting, professional lighting."
34                 ))
35             ]
36         )
37         return response.choices[0].message.content
38     except Exception as e:
39         raise RuntimeError(f"Image prompt generation error: {str(e)}")

```


Prompt: Design an eco-friendly smart water bottle for industrial workers"

Output:



Prompt: Generate a image in which a girl holding the flowers in her hands.

Output:



Sample Prompts

Prompt 1

Input: "A lightweight titanium drone frame manufactured using Selective Laser Sintering (SLS) for aerospace applications"

Prompt 2

Input: "An automated conveyor belt system for automotive assembly line using reinforced rubber belting"

Conclusion

The Multimodal Manufacturing Creator successfully demonstrates the practical integration of large language models and image generation models into a unified, user-friendly application for manufacturing concept visualization. By combining OpenAI

GPT-4o-mini for intelligent text generation with DALL·E 3 for photorealistic image synthesis, the system delivers a compelling end-to-end pipeline that transforms plain-text manufacturing ideas into professional, multi-dimensional content within seconds.

The project achieves all its stated objectives: structured narrative generation, visual prototype creation, semantic knowledge base management, and an intuitive web interface all at a minimal cost of approximately \$0.04 per generation. The deliberate choice of a single API provider simplified the architecture, reduced integration complexity, and lowered the barrier for deployment on any standard development machine.

The modular design of the codebase with separate modules for LLM handling, image generation, vector database operations, and configuration ensures maintainability and extensibility. Each component can be independently upgraded or replaced, for example swapping DALL·E 3 for Stable Diffusion or GPT-4o-mini for a future more capable model, without disrupting the rest of the system.

Key Learnings

- Prompt Engineering: Crafting focused, structured system prompts significantly improves the consistency and quality of LLM-generated content
- Two-Step Image Generation: Intermediate prompt optimization (LLM → optimized prompt → image model) produces substantially better visual results than raw user input
- Vector Databases: ChromaDB provides a practical, zero-cost solution for semantic search in small-to-medium knowledge bases without requiring cloud infrastructure
- Multimodal AI Integration: Combining text and image modalities requires careful orchestration of multiple API calls, error handling, and output synchronization

- **Cost Management:** Selecting the right model tier (GPT-4o-mini vs. GPT-4o) can reduce operational costs by 20x while maintaining acceptable quality for the use case

Streamlit for Rapid Prototyping: Streamlit enables production-quality web UIs to be built purely in Python, dramatically accelerating development time for AI applications

Future Scope & Enhancements

While the current version of the Multimodal Manufacturing Creator is fully functional and meets all project objectives, several meaningful enhancements can be pursued to extend its capabilities for real-world industrial and educational use:

1. Technical Enhancements

- **CAD File Generation:** Integrate a CAD API or parametric design library (e.g. OpenSCAD, FreeCAD) to generate basic 3D models from the generated narrative, moving beyond 2D images to actual engineering prototypes
- **Multi-Image Generation:** Allow users to generate multiple visual perspectives (front, side, cross-section) of the same concept in a single request
- **HD Image Mode by Default:** Make HD quality the default once cost is no longer a constraint, improving visual fidelity for professional presentations
- **Custom Fine-Tuned Model:** Fine-tune a smaller open-source LLM (e.g. Mistral, LLaMA) on manufacturing-specific datasets to reduce API dependency and cost
- **Streaming Responses:** Implement token streaming for the narrative to display text progressively as it is generated, improving perceived responsiveness
- **Cloud Vector Database:** Replace local ChromaDB with Pinecone or Weaviate for multi-user, cross-device concept sharing

2. Feature Enhancements

- **User Authentication:** Add login functionality to support multiple users with isolated knowledge bases and personalized history
- **PDF Report Export:** Allow users to export a complete concept report (narrative + image + metadata) as a formatted PDF document
- **Concept Comparison:** Enable side-by-side comparison of two or more generated concepts to support design decision-making.

- **Feedback & Rating System:** Allow users to rate generated outputs to build a quality feedback loop for future prompt optimization
- **Manufacturing Domain Templates:** Pre-built prompt templates for common manufacturing categories (additive manufacturing, CNC machining, injection moulding, etc.) to guide users toward better inputs
- **Multi-Language Support:** Extend the application to accept prompts and return narratives in multiple languages for global manufacturing teams

3. Deployment Enhancements

- **Cloud Deployment:** Deploy the application on platforms like Streamlit Cloud, AWS, or Azure to make it accessible without local setup
- **REST API Backend:** Convert the backend modules into a FastAPI REST service, enabling integration with third-party manufacturing tools and enterprise software
- **Mobile-Responsive UI:** Optimize the Streamlit interface for tablet and mobile access, important for shop-floor and on-site use cases
- **CI/CD Pipeline:** Set up automated testing and deployment pipelines using GitHub Actions for continuous integration

4. Research Directions

- **Multimodal Feedback Loop:** Explore using image recognition models to analyze the generated image and provide feedback to the LLM for iterative refinement of both outputs
- **Manufacturing Process Simulation:** Integrate physics-based simulation APIs to validate whether the described materials and processes are technically feasible
- **Sustainability Analysis:** Add a module that evaluates the environmental impact of the proposed materials and manufacturing processes using green manufacturing databases